VERIQO

Evidence-Qualified Intelligence OS — Enterprise Architecture and Assurance Report

Repository: `veriqo` (Go 1.24.7) · **Branch:** `claude/veriqo-enterprise-architecture-7j56b9`
Report date: 5 September 2026 · **Round 2** (supersedes the Round 1 report of the same name)

Status: SPECIFICATIONALLY IMPLEMENTED. NOT PRODUCTION QUALIFIED.

That wording is deliberate and replaces "engineering complete", which an investor can read as a conclusion and stop. Twenty gates block release; thirteen need a party that is not VERIQO. Nothing in the assurance register is above `INTERNALLY_ASSURED`.

---

0. What changed in Round 2

Round 1 built the qualification kernel and reported honestly on it. The audit that followed made a sharper point, and it was correct:

> VERIQO could constrain what it said about the world far better than what it said about itself. An assurance position lived as prose and a gate lived as a status field, so "tested by its author" and "attacked by an accredited third party" occupied the same green cell.

Round 2 closes that. Five things are new:

#	What	Why it matters
1	**Law 11 — No Self-Certification**	An assurance level requiring independent evidence is now *unrepresentable* without it
2	**The Master Assurance Graph**	Two registers became one walkable structure: gate → control → claim → evidence → validator → level → release
3	**The Independent Verification Kit**	A third party can check VERIQO **without asking VERIQO anything**
4	**The Auditor Capsule**	"Run it yourself" replaces "we say so" — and it now carries a case that can actually be run
5	**The Intelligence Fabric**	Source classes with lawfulness constraints, and vessel-behaviour analysis, so the kernel is not the whole product

Plus: readiness as five dimensions instead of a percentage; the gate lifecycle as a state machine; `ESTIMATE ≠ MEASURED ≠ VALIDATED ≠ PRODUCTION_PROVEN` as a type; the decision passport as six product kinds; and the repository hygiene the audit flagged.

59 packages · 147 files · ~47,700 lines · 814 tests · `go vet` clean · verify.sh 23/23.

---

1. Law 11, and why it is the most important thing here

> **An entity may implement and test a control, but may not unilaterally promote that control to an assurance level whose definition requires independent evidence.**

This is the assurance-layer twin of Law 7. Law 7 says an AI cannot upgrade evidence. Law 11 says an author cannot upgrade the assurance of their own control. Both are enforced the same way: the promotion is not discouraged, it is **unrepresentable**.

The ladder

```
UNDEFINED
  -> SPECIFIED
  -> IMPLEMENTED
  -> INTERNALLY_TESTED
  -> INTERNALLY_ASSURED         <- the last rung VERIQO can reach alone
  ------------------------------------------------------------
  -> READY_FOR_EXTERNAL_TEST
  -> EXTERNALLY_TESTED          <- needs a party that is not the implementer
  -> EXTERNALLY_VALIDATED
  -> OPERATIONALLY_PROVEN       <- needs a production deployment
  -> PRODUCTION_QUALIFIED       <- needs the release authority
```

No state jumps. A promotion from `INTERNALLY_ASSURED` to `PRODUCTION_QUALIFIED` is refused, and the refusal *names the rungs it skipped* — because every one of them is a question nobody answered: was it tested by somebody else, did that find anything, was it fixed, was the fix retested, has it run, for how long, under what load.

Demotion is deliberately cheap. It may skip any distance, needs no evidence, and needs only a reason. Making the honest move expensive is how organisations end up holding stale assurance.

The three attacks it refuses

1. **Internal evidence for an external rung.** Refused on evidence class.
2. **Evidence labelled external, produced by the implementer.** Refused: the validator must not be the implementer.
3. **A validator attesting to its own independence.** Refused: `AttestedBy` cannot equal the validator's own id.

---

2. The Master Assurance Graph

Gates, controls and assurance claims were three lists. They are now one graph, and a release decision walks it to the bottom:

```
GATE ──> CONTROL ──> ASSURANCE CLAIM ──> EVIDENCE ──> VALIDATOR ──> LEVEL ──> RELEASE
```

Every hop is checked. A dangling reference fails at construction. And the graph reports the quiet failure a checklist cannot see: **a control that no claim says anything about** — work that exists, is presumed fine, and has no stated property anybody could test.

What it says about VERIQO, in code rather than prose

Fact	Enforced by
Nothing is above `INTERNALLY_ASSURED`	`TestNothingInTheRegisterIsAboveInternallyAssured`
No gate is closable	`TestNoGateIsClosable`
All 20 mandatory gates rest entirely on VERIQO's own evidence	`TestEveryMandatoryGateRestsOnVeriqosOwnEvidence`
Every control has at least one claim	`TestEveryControlIsClaimedAbout`
Every unmet claim names an evidence debt with an owner	`TestEveryClaimNamesADebtOrIsFullySupported`

Each of those is a test that **fails when it stops being true**, which is the only way a statement like this stays honest for longer than one release.

Evidence Debt

"OPEN" tells a reader something is unfinished. It does not tell them what is missing, why, who would supply it, what it costs, what it blocks, or the consequence of shipping without it. Those six answers are the difference between a gap a buyer can *price* and one that reads as vagueness — and vagueness makes an honest position look evasive.

Eleven open debts. All eleven require a party that is not VERIQO.

ID	Missing	Needs
ED-001	Independent security assessment	An accredited pentest firm
ED-002	A production key root	An HSM or cloud KMS
ED-003	An external anchor for checkpoints	A timestamping authority
ED-004	Real documents, and a recovery attempt	A corpus supplier and an adversarial lab
ED-005	Independent test of the injection defence	A red team with AI-agent experience
ED-006	Operational evidence	Production infrastructure
ED-007	A live data contract and a real case	A commercial data provider
ED-008	SBOM, signing, vulnerability feed	A signing authority
ED-009	An evaluation set VERIQO did not build	An outside party
ED-010	Legal opinion on source-class lawfulness	Counsel, per jurisdiction
ED-011	Cross-implementation canonicaliser conformance	An independent RFC 8785 implementation

ED-011 was found by the register's own tests: the canonicalisation claim was short of its required level and had no debt behind it, so the gap had no owner. Every digest, ledger record, passport and replay comparison passes through that one function, and a divergence from the standard would be **invisible from inside** — the system would be perfectly self-consistent and silently unable to interoperate with anything else.

---

3. A gate is a state transition, not a checkbox

```
OPEN -> READY -> TESTING -> FINDINGS -> REMEDIATED -> RETESTED -> VALIDATED
        ^^^^^    ^^^^^^^                ^^^^^^^^^^    ^^^^^^^^
        VERIQO   needs an               VERIQO's      needs the
        alone    outside party          own work      outside party again
```

`OPEN` does not reach `VALIDATED`, however the caller spells it. The move that makes every assurance programme worthless —

```
status: OPEN     ->     status: CLOSED
```

— is refused by the type system, and the refusal names the path it skipped.

Three further rules, each closing a way the lifecycle could be gamed:

- **"Nothing was found" must be recorded explicitly.** An absent finding record and a finding-free one are the two situations a green row conflates.

- **An accepted risk needs a rationale.** It is a decision, not a fix, and the two must not look alike.
- **A gate cannot be `VALIDATED` with an open finding.**

---

4. The Independent Verification Kit

> **The verifier must not trust the system being verified.**

That rules out the obvious design. A verifier that asks VERIQO for a status has not verified, it has **relayed**. One that compares a digest to a digest VERIQO also computed has confirmed only that VERIQO is self-consistent — which a compromised system also is.

`cmd/veriqo-verify` takes a bundle and **recomputes**:

Step	What is recomputed, and why it matters
Canonicalize	Every JSON document canonicalises to a fixed point
Artefact hashes	Digests come **from the bytes**, not from the records
Signature	The payload digest is recomputed **before** the signature is checked — a verifier checking against the *supplied* digest would accept any payload at all
Provenance	Every record reaches a named origin with an ordered hop path
Ledger lineage	Rehashed **from genesis** — a chain carries its own hashes, and reading them confirms only that the file describes itself
Replay	Deterministic steps re-executed; `RECORDED` steps reported as not re-executed
Revocation	"Not checked" and "not revoked" are **different answers**
Qualification state	**Derived**, then compared with the claim

That last row is what makes it a verifier. A bundle asserting `PRODUCTION_QUALIFIED` on internal evidence is **contradicted, not believed**, and the report says: *"THESE DISAGREE. Believe the derived value."*

What it says it cannot establish, on every run

1. **Key authenticity.** A bundle produced entirely by an impostor is internally perfect. Key trust is out-of-band.
2. **Existence in time.** Without an external anchor, a hash chain proves only its own consistency, so a wholesale rewrite between two observations is invisible.
3. **Anything about evidence left out.**
4. **A defect in the canonicaliser** — when the default is used, it is the *same code* the system used, so both sides make the same mistake and agree. The `Canonicalizer` seam exists so a verifier can supply their own, and the report names which was used.

`UNVERIFIABLE` is a first-class outcome, distinct from `FAIL`. Reporting "cannot check" as "invalid" trains readers to ignore failures; reporting it as a pass is a lie.

---

5. The Auditor Capsule

What an assessor is handed so they can say *"show me"* instead of reading a document: the assurance register, the gates, the failure classes, the self-doubt register, the policy rules and authority matrix as data, the API contracts, the redaction corpus with both figures and their epistemic status, a dependency manifest, a build manifest, a threat model — **and a worked case the verifier can actually run**.

```
go run ./cmd/veriqoctl capsule ./capsule
go run ./cmd/veriqo-verify ./capsule      # every step PASSes
```

Three deliberate properties:

- **It claims exactly `INTERNALLY_ASSURED`.** Claiming less than the evidence supports costs nothing. Claiming more costs the engagement.
- **The build manifest does NOT claim verified reproducibility**, because no bit-for-bit comparison has been done. Gate G19.
- **The threat model's `not_considered` list is a required field.** A threat model that lists only what was thought about reads as complete. Ours excludes, among others: toolchain compromise, side channels, post-quantum adversaries, and a compromised operator with both commit access and release authority acting deliberately over time.

The worked case, and its most dangerous property

Without one, six of the verifier's eight steps report `UNVERIFIABLE` and the assessor is back to reading a document. With one, everything passes — **and that is the risk**. An assessor who watches every step pass can carry away the impression that VERIQO has been shown to work on data. It has been shown to work on data VERIQO wrote for the purpose.

So the capsule says the case is synthetic in four places, and its README says:

> *What it establishes about VERIQO's behaviour on REAL data: nothing. Every byte of it was written by VERIQO to be verified by you. That is evidence debt ED-007.*

The replay record deliberately contains one `DETERMINISTIC` and one `RECORDED` step. A capsule with only deterministic steps would misrepresent what replay establishes in a real case.

---

6. Four things a percentage sign hides

```
ESTIMATE  ≠  MEASURED  ≠  VALIDATED  ≠  PRODUCTION_PROVEN
```

These are not confidence levels. They name **where a number came from**, which is why they cannot be interconverted: no amount of re-running an estimate makes it a measurement, and no amount of internal measurement makes it validated.

`pkg/assurance/epistemic` makes the bare form unavailable — a `Figure` renders with its status or not at all — and a derived figure inherits the **weakest** status of its inputs. A conclusion drawn from an estimate and a measurement is an estimate.

VERIQO's own worked example: **88% weighted redaction coverage is an ESTIMATE**, because the prevalence weights are judgements and the corpus is VERIQO's own.

---

7. Readiness without a number

"VERIQO is 85% production ready" is a sentence with no truth conditions. It cannot be checked, cannot be disagreed with in detail, and invites the reader to interpolate — 85% sounds like six weeks. The reality it usually describes is a system whose architecture is finished and whose external validation has not started: two facts no single number can carry.

Dimension	Level	Who can move it
Architecture	**HIGH**	The builder
Semantics	**HIGH**	The builder
Implementation	**SUBSTANTIAL**	The builder
Production infrastructure	**NOT_STARTED**	Requires a deployment
External validation	**NOT_STARTED**	Requires an outside party

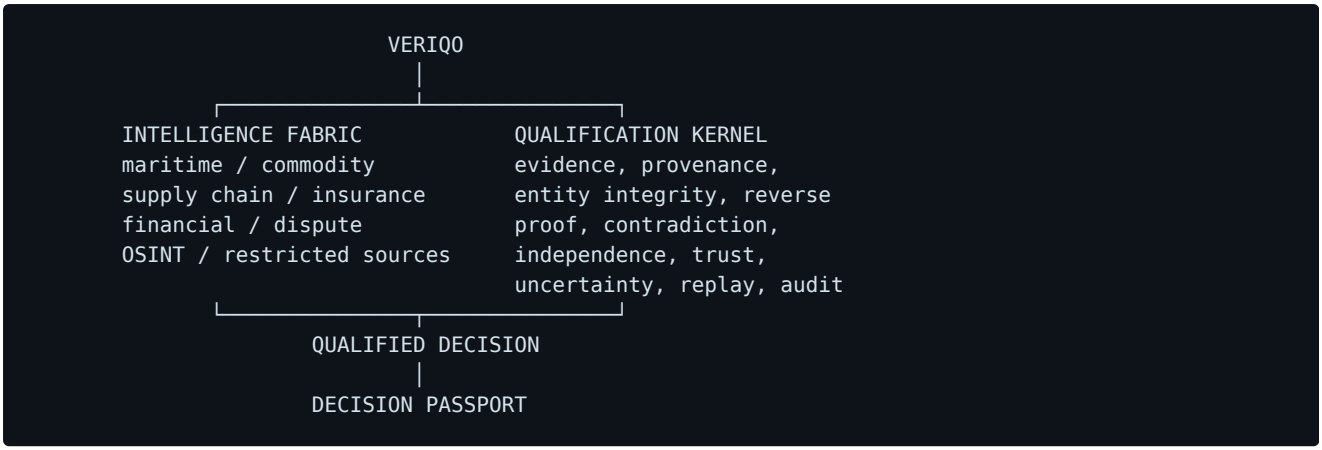
The asymmetry is the point: effort moves the first three and **cannot move the last two at all**. A dimension needing an outside party cannot be recorded above `NOT_STARTED` by an internal assessor — "we believe we would pass a pentest" is a hope, not an assessment, and a model that recorded it as one would defeat its own purpose.

There is deliberately no `Overall()`. The summary sentence is **generated from the assessments**, so it cannot drift away from them.

---

8. The Intelligence Fabric

The audit warned that VERIQO must not collapse into an evidence-governance system. The kernel is the moat; domain intelligence is the market surface. Neither is the product alone.



8.1 Source classes — including the restricted ones

Real intelligence platforms touch material ranging from a public register to a leaked corpus circulating on a hidden service. Pretending otherwise produces a system that is either useless or quietly unlawful.

The framing that matters: that range is not a quality gradient. Treating it as one loses the two things that actually decide the outcome:

- **LAWFULNESS.** A breach corpus is not low-quality. It may be unlawful to hold, and no amount of corroboration changes that.
- **ATTRIBUTION.** An anonymous paste has no producer, so Law 2 cannot be satisfied and Law 6 cannot be evaluated — two anonymous pastes might be one person.

Eleven classes, with enforced constraints. The consequential ones:

Class	May found a finding	Counts for corroboration	Permitted uses
`OFFICIAL_REGISTER`	yes	yes	screen, lead, corroborate, found, disclose
`ADVERSE_MEDIA`	**no**	**no**	screen, lead
`ANONYMOUS_DISCLOSURE`	no	no	screen, lead
`HIDDEN_SERVICE_FORUM`	no	no	screen, lead
`BREACH_DERIVED`	no	no	**screen only**

Numerousness does not substitute. Ten anonymous disclosures are still zero producers. Six outlets carrying one wire story are one source — which is the commonest way a screening system manufactures confidence.

`BREACH_DERIVED` and `HIDDEN_SERVICE_FORUM` are deliberately **separate** classes. They are routinely conflated and are not the same thing: a forum post about a vessel's movements is not stolen data, and a breach corpus on the public web is. The venue affects attribution; the acquisition affects lawfulness.

Restricted classes cannot be held **at all** without a recorded legal basis naming a jurisdiction, a purpose, and counsel. Engineering's reading of the law is not an opinion — hence ED-010.

> **This package acquires nothing.** There is no connector, crawler, credential or address in it. It is a classification and a set of refusals — the part of handling such material that belongs in code. Acquisition is a legal and operational decision made by people with names.

8.2 Vessel behaviour

Eight detectors over position reports: great-circle distance and bearing, implausible speed, reporting gaps, null positions, identifier collision, rendezvous, draught change, and reported-versus-implied speed disagreement.

Every detector answers a narrow question and refuses the broad one. It reports *"these two reports are separated by a distance no hull could cover"*, never *"this vessel spoofed its position"* — the same observation is produced by a receiver clock error, a transposed digit, two vessels sharing an identifier, and a falsification, and choosing between those is a claim about intent.

So every anomaly carries its **innocent explanations**, ordered most-mundane-first, and a **diagnostic**: what evidence would separate them. An anomaly without those is an accusation wearing the clothes of a measurement, and an analyst who sees a hundred of them learns to ignore all of them.

Three details that decide whether such a detector is usable at all:

1. **Position error is subtracted before implied speed is computed.** Two reports ten seconds apart, each accurate to half a nautical mile, imply 360 knots with the vessel stationary.
2. **0,0 is treated as the garbage value it almost always is** — an uninitialised field, a failed parse — not as a position in the Gulf of Guinea.
3. **A gap is stated as a fact about the RECORD.** "The vessel went dark" is a claim about the vessel *and about intent*. On a coastal-only network, the report leads with the receiver: offshore silence is the expected behaviour of the *receiver*.

---

9. The decision passport as a product

Nobody buys an answer. An answer from a system is worth what the system's reputation is worth, and in a dispute that is nothing — the other side's expert has an answer too. What a customer buys is a package that survives being attacked by somebody paid to attack it.

Six kinds, differing in **what they are forbidden to say**:

Kind	May not say	Because the decision belongs to
Claim qualification	"is covered", "is fraudulent"	The insurer's claims handler
Incident evidence	"was at fault", "caused the"	The investigating authority or tribunal
Quantity discrepancy	"was stolen", "the seller is liable"	The parties, or the tribunal
Collateral evidence	"is good security", "title is good"	The bank's credit committee
Dispute evidence	"the claimant will succeed"	The tribunal
Counterparty qualification	"is sanctioned", "is high risk"	The firm's compliance officer

Each forbidden statement is one the customer in that domain **will ask for and must not be given**. A system that says them is not neutral, and a neutral system that says them once is not neutral any more.

Two defects the tests found in the first version of this screen, both the classic failure of keyword filtering: inflection evaded it ("are good security" vs "is good security"), and an explicit disclaimer tripped it — flagging *"whether the policy responds is not addressed here"* would have taught authors to **delete their disclaimers**, the precise inversion of the purpose.

The check is documented as a screen, not a proof. A determined author can convey "covered" without writing the word. It makes the accidental case impossible; the real control is that one person mints and another approves.

---

10. The adversarial suite, and what it found

43 tests that assume an attacker. Each asserts the *structural* reason an attack cannot work, not that it happened not to.

Three real defects, found on the suite's first run and all now fixed:

Defect	Consequence before the fix
A ledger checksum failure *anywhere* was treated as a torn tail	Editing record 2 of 4 **silently discarded records 2, 3 and 4** and reopened the chain at height 2, with no error
The same path on a front-truncated log	The chain opened as **empty**
Every decompressed read used `io.LimitReader`	A part inflating to 256 MiB was **truncated to 64 MiB, redacted, released and marked `Verified`** — 192 MiB absent and never searched

Two of the three were in exactly the controls a marketing claim would rest on: the immutable ledger and the verified derivative.

Stated carefully: they were found by the party that wrote them, and these attacks were designed by somebody who knew where the code looked. That is the strongest available

evidence that the disproof paths are real, and it is *not the same as being attacked*. Gates G15–G18 exist for that reason, and ED-005 owns it.

The three levels of adversarial testing

```
A1  developer adversarial testing       <- VERIQO is here
A2  independent grey-box testing
A3  black-box real-world testing
```

---

11. The self-doubt register

Seventeen claims, each with a proof path **and** a disproof path — not a negative test, but an attempt to produce a counterexample. Outcomes are `ESTABLISHED`, `REFUTED`, or `UNSETTLED` (the proof succeeded and the disproof was not run — *not* established).

The register now also records **closed counterexamples**: a defect the disproof path did produce and that has since been fixed, with the fix cited.

> "We attacked this and found nothing" and "we attacked this, found a defect, and closed it" are different pieces of evidence, and the second is much stronger. A claim that has never yielded may mean the control is sound or may mean the attack was weak. A claim whose attack once succeeded is one whose attack is **known to be capable of succeeding**.

A test asserts the register as a whole: *if not one disproof path had ever produced a counterexample, either the system is perfect or the attacks are not real, and the second is far more likely.*

Deliberately `UNSETTLED`:

- **`CLAIM-REDACTION-IRREVERSIBLE`** — absence in twelve encodings is not irrecoverability, and nothing here attempts recovery.
- **`CLAIM-INJECTION-STRUCTURALLY-REFUSED`** — ten adversarial tests attack it and none has produced a counterexample, which is *precisely the weaker position*.

Every claim in this register was attacked by VERIQO and by nobody else.

---

12. Article 18 — redaction coverage

23 structural variants: 17 accepted, 6 refused by design, 0 failed.

- **Structural coverage: 74%** — the share of *variants*, **not** a pass rate. Every refusal is safe; none is a capability failure.
- **Weighted coverage: 88% (ESTIMATE)** — weighted by a *judgement* of how common each structure is.

Three of the six refused variants are COMMON in real documents — `PDF-ENCRYPTED`, `PDF-INCREMENTAL`, `PPTX-EMBEDDED-IMAGE`. A real population would land there in bulk and **the 88% would fall**.

Corpus qualification level: `L2_FIXTURE_CORRECTNESS`**.** Every fixture was built by VERIQO. Nobody has attempted to recover redacted content. *Surviving one's own corpus is the weakest form of survival available.*

The rule the worker turns on: **anything it cannot decode is refused, never reported clean.** A document nobody could read is a document nobody can certify as redacted.

---

13. The twenty gates and the nine-dimension scorecard

All twenty gates blocking. Thirteen require a party that is not VERIQO.

Dimension	Rating
Evidence integrity	YELLOW
Entity integrity	YELLOW
Reasoning integrity	YELLOW
Security	**RED**
Operational reliability	**RED**
Data rights	YELLOW
AI governance	YELLOW
Replayability	YELLOW
External validation	**RED**

Nothing is GREEN. Release refused, with nine reasons. There is no aggregate score, deliberately: a single figure lets a strong dimension carry a weak one, and the weak one is what a customer needs to see.

`EXTERNAL_VALIDATION`'s RED bounds every other rating. *No party outside VERIQO has examined, attacked, validated or corroborated any part of this system.*

---

14. What is deliberately absent

- **An aggregate confidence score** — nine uncertainty dimensions, no `Overall()`.
- **A readiness percentage** — five dimensions, no aggregate.
- **A `CONFIRMED` verdict** — evidence can fail to disconfirm; it cannot confirm.

- **A merge operation** — five outcomes, and a merge needs a reviewer.
- **A trust path from source to conclusion.**
- **A production KMS** — `SoftwareRoot` is a test double and refuses production mode.
- **An anchor implementation** — an anchor VERIQO controls proves that VERIQO agrees with itself. The interface is declared and left unimplemented, and the register records the absence as a *decision* so it does not read as an oversight.
- **`time.Now()` on any deterministic path** — enforced by an architecture test.
- **Any acquisition of restricted-class material** — no connector, no crawler, no credential.

---

15. Verification

```
./scripts/verify.sh    # 23 passed, 0 failed, 15 explicitly NOT run
```

The honesty checks fail the build if the system starts overstating itself:

- the scorecard refuses its own release
- no gate is satisfied
- **nothing is above** `INTERNALLY_ASSURED`
- **every mandatory gate rests on VERIQO alone**
- coverage carries the word `ESTIMATE`
- readiness offers no aggregate figure
- both external readiness dimensions are `NOT_STARTED`
- every evidence debt has an owner *and* a risk
- **`veriqo-verify` passes every step on a freshly built capsule** — and fails the build if any step is unverifiable

Fifteen things it explicitly does **not** run, each tied to a gate or a debt — including `independent canonicaliser` (ED-011), `external anchor check` (ED-003), and `independent red team` (G15–G18).

---

16. The honest summary

> Architecture high, semantics high, implementation substantial. Production infrastructure not started, external validation not started. The weakest dimension is production infrastructure, and no single figure is offered because a strong dimension must not be allowed to carry a weak one.

Or, for a customer asking whether this can be trusted:

> **VERIQO's core qualification kernel is internally assured. Production deployment remains gated by independently verifiable security,**

> **operational, data-rights and external-validation evidence — eleven named debts, every one of which requires a party that is not VERIQO.**

The engineering problem is largely closed. The **qualification** problem is now the bottleneck, and it cannot be closed by writing more Go. That is not a failure of this report; it is its principal finding — and a platform that reports it accurately about itself is the only kind that can be trusted to report it about anything else.

Every figure regenerable: `go run ./cmd/veriqoctl all`*. Every claim checkable without trusting this document:* `go run ./cmd/veriqoctl capsule ./c && go run ./cmd/veriqo-verify ./c`*.*